Python & Algorithms for Coding Interviews

Section 5 — Universal Code Templates

Every template here is short enough to memorise. Each one is a battle-tested starting point — adapt variable names and logic to the specific problem, but the skeleton stays the same.

5.1 DFS — Recursive (Tree & Graph)

```
# Tree DFS
def dfs_tree(root):
    if not root:
        return 0   # adjust return type per problem
    left  = dfs_tree(root.left)
    right = dfs_tree(root.right)
    return 1 + max(left, right)   # example: tree height

# Graph DFS
def dfs_graph(graph, node, visited):
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs_graph(graph, nb, visited)

# Outer loop for disconnected graphs
visited = set()
for node in range(n):
    if node not in visited:
        dfs_graph(graph, node, visited)
```

✓ **Solves**	LeetCode 104 Max Depth • 200 Number of Islands • 543 Diameter • 133 Clone Graph

5.2 DFS — Iterative (Explicit Stack)

```
def dfs_iterative(graph, start):
    visited = set()
    stack   = [start]
    while stack:
```

```
        node = stack.pop()
        if node in visited: continue
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                stack.append(nb)

# Use when recursion depth might exceed Python's 1000 frame limit
```

✓Solves	LeetCode 200 Number of Islands • 695 Max Area • Any deep graph problem

5.3 BFS — Standard & Multi-Source

```
from collections import deque

# Standard BFS — shortest path
def bfs(graph, start):
    visited = set([start])
    queue   = deque([(start, 0)])   # (node, distance)
    while queue:
        node, dist = queue.popleft()
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))

# Level-order tree BFS
def level_order(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):   # one level at a time
            node = queue.popleft()
            level.append(node.val)
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result

# Multi-source BFS (grid) — enqueue ALL sources at start
def multi_bfs(grid):
    rows, cols = len(grid), len(grid[0])
    queue, visited = deque(), set()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:   # source condition
                queue.append((r,c,0)); visited.add((r,c))
    dirs = [(0,1),(0,-1),(1,0),(-1,0)]
    while queue:
        r,c,dist = queue.popleft()
        for dr,dc in dirs:
            nr,nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited:
```

```
                visited.add((nr,nc)); queue.append((nr,nc,dist+1))
```

✓Solves	LeetCode 102 Level Order • 127 Word Ladder • 994 Rotting Oranges • 542 01 Matrix

5.4 Binary Search — 3 Variants

```
# Variant 1: Find exact target
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:              # note: <=
        mid = left + (right - left) // 2
        if   nums[mid] == target: return mid
        elif nums[mid] <  target: left  = mid + 1
        else:                     right = mid - 1
    return -1

# Variant 2: Left boundary (first index where nums[i] >= target)
def left_bound(nums, target):
    left, right = 0, len(nums)        # note: right = len
    while left < right:               # note: <
        mid = (left + right) // 2
        if nums[mid] < target: left  = mid + 1
        else:                  right = mid
    return left

# Variant 3: Binary search on answer space
def search_on_answer(nums, k):
    left, right = min(nums), sum(nums)  # define answer range
    while left < right:
        mid = (left + right) // 2
        if feasible(nums, k, mid): right = mid
        else:                      left  = mid + 1
    return left
```

✓Solves	LeetCode 704 • 278 First Bad Version • 875 Koko Eating Bananas • 410 Split Array Largest Sum

5.5 Backtracking — Subsets / Combos / Permutations

```
# Subsets
def subsets(nums):
    result = []
    def bt(start, path):
        result.append(list(path))    # record at every node
        for i in range(start, len(nums)):
            path.append(nums[i]); bt(i+1, path); path.pop()
    bt(0, []); return result
```

```
# Combinations (choose k from n)
def combine(n, k):
    result = []
    def bt(start, path):
        if len(path)==k: result.append(list(path)); return
        for i in range(start, n+1):
            path.append(i); bt(i+1, path); path.pop()
    bt(1, []); return result

# Permutations
def permutations(nums):
    result = []
    def bt(path, remaining):
        if not remaining: result.append(list(path)); return
        for i in range(len(remaining)):
            path.append(remaining[i])
            bt(path, remaining[:i]+remaining[i+1:])
            path.pop()
    bt([], nums); return result

# Skip duplicates (sort first + skip same siblings)
nums.sort()
if i > start and nums[i] == nums[i-1]: continue
```

✓Solves	LeetCode 78 Subsets • 77 Combinations • 46 Permutations • 90 Subsets II • 39 Combination Sum

5.6 DP — Top-Down Memoisation

```
from functools import lru_cache

# 1D DP
@lru_cache(maxsize=None)
def dp_1d(i):
    if i <= 0: return BASE_CASE
    return f(dp_1d(i-1), dp_1d(i-2))   # adjust recurrence

# 2D DP
@lru_cache(maxsize=None)
def dp_2d(i, j):
    if i==0 or j==0: return BASE_CASE
    if s1[i-1]==s2[j-1]: return dp_2d(i-1, j-1) + 1
    return max(dp_2d(i-1, j), dp_2d(i, j-1))

# Note: list inputs must be converted to tuple for hashability
@lru_cache(maxsize=None)
def dp_coins(amount, coins):  # coins must be tuple
    if amount==0: return 0
    if amount <0: return float('inf')
    return 1 + min(dp_coins(amount-c, coins) for c in coins)

dp_2d.cache_clear()   # reset between test cases if needed
```

✓Solves	LeetCode 70 Climbing Stairs • 322 Coin Change • 1143 LCS • 139 Word Break

5.7 DP — Bottom-Up Tabulation

```
# 1D DP
def dp_1d_tab(n):
    dp    = [0] * (n + 1)
    dp[0] = BASE_0
    dp[1] = BASE_1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]   # adjust recurrence
    return dp[n]

# 2D DP (LCS / Edit Distance)
def dp_2d_tab(s1, s2):
    m, n = len(s1), len(s2)
    dp   = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1]==s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1
            else:                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

# Space-optimised rolling array
def dp_rolling(s1, s2):
    n  = len(s2)
    dp = [0]*(n+1)
    for ch in s1:
        prev = 0
        for j in range(1, n+1):
            temp  = dp[j]
            dp[j] = prev+1 if ch==s2[j-1] else max(dp[j], dp[j-1])
            prev  = temp
    return dp[n]
```

✓Solves	LeetCode 322 Coin Change • 1143 LCS • 300 LIS • 72 Edit Distance • 416 Partition Equal Subset Sum

5.8 Union Find (DSU)

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank   = [0] * n
        self.count  = n

    def find(self, x):                        # path compression
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
```

```
    def union(self, x, y):                    # union by rank
        px, py = self.find(x), self.find(y)
        if px == py: return False
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        self.count -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)

uf = UnionFind(n)
for u, v in edges: uf.union(u, v)
print(uf.count)   # number of connected components
```

✓Solves	LeetCode 547 Number of Provinces • 684 Redundant Connection • 721 Accounts Merge

5.9 Trie (Prefix Tree)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end   = False

class Trie:
    def __init__(self): self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children: return False
            node = node.children[ch]
        return node.is_end

    def starts_with(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children: return False
            node = node.children[ch]
        return True
```

✓Solves	LeetCode 208 Implement Trie • 212 Word Search II • 648 Replace Words • 677 Map Sum Pairs

5.10 Graph Build + Traversal

```
from collections import defaultdict

# Unweighted undirected
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)   # omit for directed

# Weighted
for u, v, w in weighted_edges:
    graph[u].append((v, w))
    graph[v].append((u, w))

# Count connected components
visited, count = set(), 0
def dfs(node):
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited: dfs(nb)
for i in range(n):
    if i not in visited:
        dfs(i); count += 1
```

✓Solves	LeetCode 200 Number of Islands • 547 Number of Provinces • 323 Number of Connected Components

5.11 Heap — Min / Max / Priority Queue

```
import heapq

heap = []
heapq.heappush(heap, val)          # O(log n)
smallest = heapq.heappop(heap)     # O(log n)
peek     = heap[0]                 # O(1)
heapq.heapify(list_of_nums)        # O(n)

# Max-heap — negate values
heapq.heappush(max_heap, -val)
largest = -heapq.heappop(max_heap)

# Priority queue with metadata
heapq.heappush(heap, (priority, data))
priority, data = heapq.heappop(heap)
```

```
# Top K in a stream — maintain size-k min-heap
for num in stream:
    heapq.heappush(heap, num)
    if len(heap) > k: heapq.heappop(heap)
# heap now contains the k largest
```

✓Solves	LeetCode 215 Kth Largest • 347 Top K Frequent • 295 Find Median from Stream • 743 Network Delay

5.12 Sliding Window — Fixed & Variable

```
# Fixed window
def fixed window(nums, k):
    window_sum = sum(nums[:k])
    result     = [window_sum]
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i-k]
        result.append(window_sum)
    return result

# Variable window — longest satisfying condition
def variable_window(s):
    left, best, state = 0, 0, {}
    for right, ch in enumerate(s):
        state[ch] = state.get(ch, 0) + 1   # expand
        while invalid(state):               # shrink
            state[s[left]] -= 1
            if state[s[left]] == 0: del state[s[left]]
            left += 1
        best = max(best, right - left + 1)  # record
    return best
```

✓Solves	LeetCode 3 Longest Substring • 76 Minimum Window Substring • 567 Permutation in String

5.13 Two Pointers

```
# Opposite ends — sorted array
def two_ptr_opposite(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if   total == target: return [left, right]
        elif total <  target: left  += 1
        else:                 right -= 1
    return []

# Same direction — partition / remove
def two_ptr_same(nums):
```

```
    slow = 0
    for fast in range(len(nums)):
        if condition(nums[fast]):
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

✓Solves	LeetCode 167 Two Sum II • 15 3Sum • 11 Container With Water • 26 Remove Duplicates

5.14 Monotonic Stack

```
# Next Greater Element — decreasing stack
def next_greater(nums):
    result = [-1] * len(nums)
    stack  = []
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
            result[stack.pop()] = num
        stack.append(i)
    return result

# Largest Rectangle in Histogram — increasing stack
def largest_rect(heights):
    stack, max_area = [], 0
    heights = heights + [0]   # sentinel to flush stack
    for i, h in enumerate(heights):
        start = i
        while stack and heights[stack[-1]] > h:
            idx      = stack.pop()
            width    = i - (stack[-1] if stack else -1) - 1
            max_area = max(max_area, heights[idx]*width)
            start    = idx
        stack.append(start)
    return max_area
```

✓Solves	LeetCode 739 Daily Temperatures • 496 Next Greater • 84 Largest Rectangle • 42 Trapping Rain Water

5.15 Topological Sort — Kahn's BFS

```
from collections import defaultdict, deque

def topo_sort(n, prerequisites):
    graph  = defaultdict(list)
    in_deg = [0] * n
    for a, b in prerequisites:   # b must come before a
        graph[b].append(a)
        in_deg[a] += 1
```

```
    queue = deque(i for i in range(n) if in deg[i] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph[node]:
            in_deg[nb] -= 1
            if in_deg[nb] == 0: queue.append(nb)

    return order if len(order)==n else []  # [] = cycle detected
```

✓Solves	LeetCode 207 Course Schedule • 210 Course Schedule II • 269 Alien Dictionary

5.16 Dijkstra's Shortest Path

```
import heapq
from collections import defaultdict

def dijkstra(n, edges, start):
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
        graph[v].append((u, w))   # undirected

    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)]

    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue  # stale entry — skip
        for nb, w in graph[node]:
            nd = dist[node] + w
            if nd < dist[nb]:
                dist[nb] = nd
                heapq.heappush(heap, (nd, nb))

    return dist
```

✓Solves	LeetCode 743 Network Delay • 1631 Path Min Effort • 787 Cheapest Flights • 1514 Max Probability

5.17 Interval — Merge, Insert, Count Rooms

```
# Merge overlapping intervals
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
```

```
    for s, e in intervals[1:]:
        if s <= merged[-1][1]: merged[-1][1] = max(merged[-1][1], e)
        else:                  merged.append([s, e])
    return merged

# Minimum meeting rooms needed
import heapq
def min_rooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []
    for s, e in intervals:
        if heap and heap[0] <= s: heapq.heapreplace(heap, e)
        else:                     heapq.heappush(heap, e)
    return len(heap)

# Remove minimum intervals to eliminate all overlaps
def erase_overlap(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by end
    count, last_end = 0, float('-inf')
    for s, e in intervals:
        if s >= last_end: last_end = e
        else:             count += 1
    return count
```

✓Solves	LeetCode 56 Merge • 57 Insert • 253 Meeting Rooms II • 435 Non-overlapping • 452 Min Arrows

5.18 Matrix DFS / BFS

```
dirs = [(0,1),(0,-1),(1,0),(-1,0)]  # 4-directional

# DFS flood fill
def dfs_grid(grid, r, c, visited):
    rows, cols = len(grid), len(grid[0])
    if r<0 or r>=rows or c<0 or c>=cols: return
    if (r,c) in visited or grid[r][c]==BLOCKED: return
    visited.add((r,c))
    for dr,dc in dirs:
        dfs_grid(grid, r+dr, c+dc, visited)

# BFS shortest path
from collections import deque
def bfs grid(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    visited = set([start])
    queue   = deque([(start[0], start[1], 0)])
    while queue:
        r,c,dist = queue.popleft()
        if (r,c)==end: return dist
        for dr,dc in dirs:
            nr,nc = r+dr, c+dc
            if (0<=nr<rows and 0<=nc<cols
                and grid[nr][nc]!=BLOCKED
                and (nr,nc) not in visited):
```

```
                visited.add((nr,nc))
                queue.append((nr,nc,dist+1))
    return -1
```

✓Solves	LeetCode 200 Number of Islands • 994 Rotting Oranges • 1091 Shortest Path Binary Matrix • 695 Max Area

---

End of Section 5. Continue with Section 6 — Time Complexity Cheat Sheet.